

Team: Team 08 - G8 DatingApp AI66B

Members: Lê Duy Quyền, Nguyễn Sơn Hải, Nguyễn Minh Quang

Repository: <https://github.com/SE-FDA-NEU-AI66B/G8-DatingApp-AI66B>

Pull Request: <https://github.com/SE-FDA-NEU-AI66B/G8-DatingApp-AI66B/pull/7>

Merge commit: aeaadf84bdc9d26c66c40c9834003814f2b253a3

Submitted by: hai291

Process Dossier

1. Chosen process and its position on the spectrum

We will follow an incremental process with agile sprints and fixed milestone gates. One cycle starts with a short planning meeting where the team reviews the GitHub project board, selects issues, and agrees on acceptance criteria. Developers work on feature branches, ask for design feedback when the work affects the interface, and submit pull requests. Each change is reviewed by another member, tested locally, and merged only when the agreed checks pass. At the end of the cycle, the team updates documentation and demonstrates the working increment. The output is a tested slice of the DatingApp, updated board status, and any changed setup or process notes.

Our position is about 75% agile and 25% plan-driven. The sprint backlog, UI details, test cases, and implementation order can be reopened every cycle because the current backlog shows uncertainty around authentication, database work, documentation, and testing. The semester milestones, final demo date, repository workflow, core project scope, and requirement to submit reviewed work are fixed for the whole semester. This gives us room to adapt while respecting course checkpoints.

2. The five diagnostic questions

Our requirements are volatile rather than stable. The project is still evolving from a small Leptos/Rust web application into a more complete DatingApp, and the visible TODO list includes major unfinished areas such as user authentication, database support, tests, and documentation. Those items can change after feedback because they affect both user behavior and technical architecture.

The project has low safety impact and limited legal impact. A semester dating application does not control physical systems or make high-risk decisions, so it does not require heavyweight safety certification. Because dating applications can involve personal profiles and credentials, we still need basic documentation for authentication, privacy-sensitive data, and changes that affect stored user information.

The team is small and working in the same course context, so communication cost is moderate. A small team can resolve questions through short meetings, chat, and pull request comments instead of formal handoff documents. That supports an agile rhythm, but review comments remain important because members may work at different times.

The customer can engage at fixed checkpoints more reliably than continuously. The instructor will assess milestone submissions and the final demo, while any real users we consult may only be available for brief feedback on screens or flows. We will collect feedback at each milestone and sprint review, then convert it into prioritized GitHub issues.

The course culture and contract constraints allow iteration inside four fixed milestones. We can choose our internal process, but the deadline structure requires visible progress, reviewed repository history, and a final demo. Before submission, the repository will be public or the instructor account will be added as a collaborator. One process file will be maintained for the team.

3. Critical thinking: risks of the opposite choice

If we moved to a fully plan-driven process, the biggest risk would be freezing the design before we understand the authentication, database, and user experience details well enough. The mechanism is delay: the team would write a complete specification, then discover during implementation or instructor feedback that the planned flows do not fit the actual app. The first symptom would be a milestone where the repository contains documents and partially connected code, but no demonstrable end-to-end feature.

4. Process rules your team commits to

- Sprint length is two weeks; the GitHub project board is reprioritized at the start of each sprint.
- Every change reaches main through a pull request reviewed by at least one other team member.
- Each pull request must include either a test result, a screenshot, or a short explanation of why the change is documentation-only.
- Requirement changes after sprint planning are recorded as GitHub issues before implementation starts.
- Milestone branches are merged at least 12 hours before the course deadline unless the team records the reason in the pull request.